

IFT 3911 – Devoir 1

Contraintes OCL

1 Volet administratif

1.1 Codes des lieux de transport

Un aéroport, un port ou une gare est identifié par un code de trois lettres. Le code est unique parmi les lieux de transport du même type.

```
context LieuTransport

inv CodeTroisLettres:
  self.code.size() = 3
  and
  'ABCDEFGHJKLMNOPQRSTUVWXYZ'
    .indexOf(self.code.substring(1,1).toUpperCase()) > 0
  and
  'ABCDEFGHJKLMNOPQRSTUVWXYZ'
    .indexOf(self.code.substring(2,2).toUpperCase()) > 0
  and
  'ABCDEFGHJKLMNOPQRSTUVWXYZ'
    .indexOf(self.code.substring(3,3).toUpperCase()) > 0

inv CodeUniqueParType:
  LieuTransport.allInstances()->forall(1 |
    (1 <> self and 1.ocltType() = self.ocltType())
    implies
    1.code <> self.code
  )
```

1.2 Codes des exploitants

Une compagnie aérienne est identifiée par un code de deux lettres. Une compagnie de croisière ou une ligne de train possède un code de moins de six caractères.

Le code est unique parmi les exploitants du même type.

```
context Exploitant

inv FormatCode:
  if self.ocltIsKindOf(CompagnieAerienne) then
    self.code.size() = 2
    and
    'ABCDEFGHJKLMNOPQRSTUVWXYZ'
      .indexOf(self.code.substring(1,1).toUpperCase()) > 0
    and
    'ABCDEFGHJKLMNOPQRSTUVWXYZ'
      .indexOf(self.code.substring(2,2).toUpperCase()) > 0
  else
```

```

        self.code.size() >= 1
        and self.code.size() < 6
    endif

inv CodeUniqueParType:
    Exploitant.allInstances()->forAll(e |
        (e <> self and e.oclType() = self.oclType())
        implies
        e.code <> self.code
    )

```

1.3 Véhicule appartenant au bon exploitant

Chaque vol doit être assigné à un avion appartenant à la compagnie aérienne qui exploite le vol.

```

context Vol

inv AvionDeLaCompagnie:
    self.avion.compagnieAerienne
    =
    self.compagnieAerienne

```

De manière analogue, un trajet utilise un train exploité par la ligne de train qui fournit ce trajet.

```

context Trajet

inv TrainDeLaLigne:
    self.train.ligneTrain
    =
    self.ligneTrain

```

De même, un itinéraire utilise un paquebot exploité par la compagnie de croisière qui fournit cet itinéraire.

```

context Itineraire

inv PaquebotDeLaCompagnie:
    self.paquebot.compagnieCroisiere
    =
    self.compagnieCroisiere

```

1.4 Absence de chevauchement des voyages

Deux vols assignés au même avion ne peuvent pas se chevaucher.

```

context Avion

inv VolsNonChevauchants:
    self.vol->forAll(v1, v2 |
        v1 <> v2 implies
        (
            v1.dateHeureArrivee <= v2.dateHeureDepart
            or
            v2.dateHeureArrivee <= v1.dateHeureDepart
        )
    )

```

Deux itinéraires assignés au même paquebot ne peuvent pas se chevaucher.

```
context Paquebot

inv ItinerairesNonChevauchants:
  self.itineraire->forAll(i1, i2 |
    i1 <> i2 implies
    (
      i1.dateHeureArrivee <= i2.dateHeureDepart
      or
      i2.dateHeureArrivee <= i1.dateHeureDepart
    )
  )
)
```

Les mêmes règles s'appliquent aux trajets effectués par un même train.

```
context Train

inv TrajetsNonChevauchants:
  self.trajet->forAll(t1, t2 |
    t1 <> t2 implies
    (
      t1.dateHeureArrivee <= t2.dateHeureDepart
      or
      t2.dateHeureArrivee <= t1.dateHeureDepart
    )
  )
)
```

1.5 Identifiant d'un vol

L'identifiant d'un vol est constitué du code à deux lettres de la compagnie aérienne suivi exactement de quatre chiffres. L'identifiant complet est unique à chaque vol.

```
context Vol

inv FormatNumeroVol:
  self.numero.size() = 6

  and
  self.numero.substring(1,2)
    = self.compagnieAerienne.code

  and
  '0123456789'
    .indexOf(self.numero.substring(3,3)) > 0

  and
  '0123456789'
    .indexOf(self.numero.substring(4,4)) > 0

  and
  '0123456789'
    .indexOf(self.numero.substring(5,5)) > 0

  and
  '0123456789'
    .indexOf(self.numero.substring(6,6)) > 0

inv NumeroVolUnique:
```

```

Vol.allInstances()->forall(v |
  v <> self
  implies
  v.numero <> self.numero
)

```

1.6 Sections d'un avion

Une même section ne peut apparaître qu'une seule fois dans un avion et tout avion doit contenir une section Économique.

```

context Avion

inv SectionsUniques:
  self.sectionAvion->isUnique(s | s.type)

inv SectionEconomiqueObligatoire:
  self.sectionAvion->exists(s |
    s.type = TypeSectionAvion::ECONOMIQUE
  )

```

1.7 Durée d'un itinéraire

Un itinéraire ne peut pas durer plus de quatorze jours. On suppose que l'opération `getDuree()` retourne la durée de l'itinéraire en jours.

```

context Itineraire

inv DureeMaximum:
  self.getDuree() <= 14

```

1.8 Ports d'un itinéraire

Un itinéraire doit visiter au moins trois ports distincts.

Dans le modèle, l'ordre des ports est représenté à l'aide des `EscalePortuaire`. Chaque escale possède un attribut `ordre` et est associée à un port.

```

context Itineraire

inv AuMoinsTroisPortsDistincts:
  self.escalePortuaire.port
    ->asSet()
    ->size() >= 3

```

L'ordre attribué aux escales doit être unique dans un itinéraire.

```

context Itineraire

inv OrdreEcalesUnique:
  self.escalePortuaire
    ->isUnique(e | e.ordre)

```

Le port de départ et le port d'arrivée doivent être le même.

```

context Itineraire

inv MemePortDepartEtArrivee:

```

```

self.escalePortuaire
  ->sortedBy(e | e.ordre)
  ->first().port
=
self.escalePortuaire
  ->sortedBy(e | e.ordre)
  ->last().port

```

1.9 Gares d'un trajet

Un trajet comporte au moins deux gares et une même gare ne peut pas apparaître plus d'une fois dans le trajet.

```

context Trajet

inv GaresDistinctes:
  self.gare->size() >= 2
  and
  self.gare->size()
    =
  self.gare->asSet()->size()

```

2 Volet client

2.1 Réservations confirmées d'un passager

Un passager peut avoir plusieurs réservations confirmées pour des voyages différents, mais il ne peut pas avoir plus d'une réservation confirmée pour un même voyage.

```

context Passager

inv UneReservationConfirmerParVoyage:
  self.reservation
    ->select(r |
      r.statut = StatutReservation::CONFIRMEE
    )
    ->isUnique(r | r.voyage)

```

2.2 Réservation d'un siège d'avion

La réservation d'un siège d'avion est possible uniquement si le vol existe et qu'il possède au moins un siège disponible dans la classe demandée.

```

context ControleurReservation::reserverSiege(
  numeroVoyage : String,
  priorite : PrioriteSiege,
  classe : TypeSectionAvion
) : ReservationSiege

pre VolEtSiegeDisponibles:
  Vol.allInstances()->exists(v |
    v.numero = numeroVoyage
    and
    v.getSiegesDisponibles(classe)->notEmpty()
  )

```

```

post ReservationDuBonVoyage:
  result.voyage.numero = numeroVoyage
  and
  result.statut = StatutReservation::RESERVEE

post SiegeDeLaBonneClasse:
  result.siege.sectionSieges
    .oclAsType(SectionAvion).type
  =
  classe

post SiegeNonDoublementReserve:
  ReservationSiege.allInstances()
    ->select(r |
      r.voyage = result.voyage
      and
      r.siege = result.siege
      and
      (
        r.statut = StatutReservation::CONFIRMEE
        or
        (
          r.statut = StatutReservation::RESERVEE
          and
          not r.estExpiree()
        )
      )
    )
    ->size() = 1

```

2.3 Réserveation d'un siège de train

La même règle s'applique à la réserveation d'un siège pour un trajet ferroviaire.

```

context ControleurReservation::reserverSiege(
  numeroVoyage : String,
  priorite : PrioriteSiege,
  classe : TypeSectionTrain
) : ReservationSiege

pre TrajetEtSiegeDisponibles:
  Trajet.allInstances()->exists(t |
    t.numero = numeroVoyage
    and
    t.getSiegesDisponibles(classe)->notEmpty()
  )

post ReservationDuBonVoyage:
  result.voyage.numero = numeroVoyage
  and
  result.statut = StatutReservation::RESERVEE

post SiegeDeLaBonneClasse:
  result.siege.sectionSieges
    .oclAsType(SectionTrain).type
  =
  classe

```

```

post SiegeNonDoublementReserve:
  ReservationSiege.allInstances()
  ->select(r |
    r.voyage = result.voyage
    and
    r.siege = result.siege
    and
    (
      r.statut = StatutReservation::CONFIRMEE
      or
      (
        r.statut = StatutReservation::RESERVEE
        and
        not r.estExpiree()
      )
    )
  )
  ->size() = 1

```

2.4 Réserve d'une cabine

Une cabine ne peut être réservée que s'il existe une cabine disponible dans la classe demandée pour l'itinéraire sélectionné.

```

context ControleurReservation::reserverCabine(
  numeroItineraire : String,
  classe : TypeSectionPaquebot
) : ReservationCabine

pre ItineraireEtCabineDisponibles:
  Itineraire.allInstances()->exists(i |
    i.numero = numeroItineraire
    and
    i.getCabinesDisponibles(classe)->notEmpty()
  )

post ReservationDuBonItineraire:
  result.voyage.numero = numeroItineraire
  and
  result.statut = StatutReservation::RESERVEE

post CabineDeLaBonneClasse:
  result.cabine.sectionPaquebot.type
  =
  classe

post CabineNonDoublementReservee:
  ReservationCabine.allInstances()
  ->select(r |
    r.voyage = result.voyage
    and
    r.cabine = result.cabine
    and
    (
      r.statut = StatutReservation::CONFIRMEE
      or
      (
        r.statut = StatutReservation::RESERVEE

```

```

        and
        not r.estExpiree()
    )
)
)
->size() = 1

```

2.5 Paiement d'une réservation

Le paiement est possible uniquement si la réservation identifiée par le numéro fourni existe, est toujours réservée et n'est pas expirée.

Après le paiement, la réservation est confirmée, associée au passager et associée au paiement produit par l'opération.

La même contrainte s'applique aux réservations de sièges et de cabines.

```

context ControleurPaiement::payer(
    numeroReservation : String,
    noms : String,
    courriel : String,
    numeroPasseport : String,
    carteCredit : String
) : Paiement

pre ReservationValide:
    Reservation.allInstances()->exists(r |
        r.numeroReservation = numeroReservation
        and
        r.statut = StatutReservation::RESERVEE
        and
        not r.estExpiree()
    )

post ReservationConfirme:
    Reservation.allInstances()
    ->select(r |
        r.numeroReservation = numeroReservation
    )
    ->forAll(r |
        r.statut = StatutReservation::CONFIRMEE
    )

post PassagerAssocie:
    Reservation.allInstances()
    ->select(r |
        r.numeroReservation = numeroReservation
    )
    ->forAll(r |
        r.passager.noms = noms
        and
        r.passager.courriel = courriel
        and
        r.passager.numeroPasseport = numeroPasseport
    )

post PaiementAssocie:
    Reservation.allInstances()
    ->select(r |

```

```
        r.numeroReservation = numeroReservation
    )
    ->forall(r |
        r.paielement = result
    )
```